

Java Avancé

Devoir sur ordinateur

Design Patterns · Génériques · Streams · Réflexion · I/O · Regex

DUREE
4 heures

NIVEAU
Bachelor 2

DOCS
Autorisés

RENDU
Git + ZIP

COMMENT RENDRE

- **Un projet Maven/Gradle** par partie. La structure est fournie.
- **REPONSES.md** à la racine — toutes les réponses théoriques y vont. Une section par exercice.
- **Rendu** : ZIP nommé `NOM_Prenom_B2_Java.zip` + push sur ton dépôt Git. Lien dans REPONSES.md.
- **Tests** : lance `mvn test` pour valider chaque partie avant de rendre.
- **Interdit** : IA générative, copier-coller entre étudiants.

RECAPITULATIF

01	Design Patterns	35 pts
02	Génériques, Lambdas & Streams	30 pts
03	Réflexion & Annotations	25 pts
04	I/O, Regex & REST	10 pts

PARTIE 01 Design Patterns*Singleton · Builder · Decorator · Factory Method · Bridge · State · Chain of Responsibility* — **35 points**

Contexte général. Tu travailles sur un jeu de rôle en ligne. Toutes les parties de cette section utilisent les classes de ce jeu comme support.

A.**Singleton — le serveur de jeu [5 pts]**

Il ne doit exister qu'une seule instance du serveur de jeu dans toute l'application.

`src/main/java/partiel/GameServer.java`

```
public class GameServer {
    private final int port;
    private int connectedPlayers = 0;

    // 1. Implémente le pattern Singleton avec Double-Checked Locking. (3 pts)
    //    Le port est fixé à 8080 à la création.

    // 2. Méthode connect() : incrémente connectedPlayers. (1 pt)
    // 3. Méthode getConnectedPlayers() : retourne le nombre de joueurs. (1 pt)
}
```

REPONSES.MD §1.A

1. Pourquoi le mot-clé volatile est-il indispensable sur l'instance dans le Double-Checked Locking ?
2. Donne une alternative plus simple et tout aussi thread-safe en Java. Pourquoi est-elle préférable ?

B.**Builder — créer un personnage [6 pts]**

On veut créer des personnages de jeu avec de nombreuses propriétés optionnelles. Le constructeur classique devient vite illisible.

`src/main/java/partiel/Personnage.java`

```
public class Personnage {
    private final String nom;           // obligatoire
    private final String classe;       // obligatoire (ex: Guerrier, Mage, Voleur)
    private int pv;                     // défaut : 100
}
```

```
private int mana;           // défaut : 50
private String arme;       // défaut : "Poings"
private boolean estElite;  // défaut : false

// Constructeur privé – uniquement accessible via le Builder.

// Implémente la classe statique interne Personnage.Builder.           (6 pts)
// Chaque setter retourne le Builder (chaining).
// build() vérifie que nom et classe ne sont pas null/vides,
// sinon lance IllegalArgumentException.
// toString() sur Personnage affiche tous les champs.
}
```

C.

Factory Method — forger une arme [5 pts]

Contexte. Un `Forgeron` (classe abstraite) sait forger une arme, mais c'est la sous-classe qui décide laquelle.

`src/main/java/partiel/Forgeron.java`

```
public interface Arme {
    String nom();
    int degats();
}

public abstract class Forgeron {
    // Méthode factory à implémenter par les sous-classes.
    public abstract Arme forger();

    // Méthode finale : affiche l'arme forgée.           (1 pt)
    public final String presenterArme() { ... }
}

// Implémente : ForgeronEpee (Epee : 80 dégâts)       (2 pts)
// Implémente : ForgeronArc (Arc : 60 dégâts)         (2 pts)
```

D.

Decorator — améliorer une potion [6 pts]

Contexte. Une `PotionDeBase` restaure 50 PV. On veut lui ajouter des effets supplémentaires par composition.

`src/main/java/partiel/Potion.java`

```
public interface Potion {
    int getPv();           // points de vie restaurés
    String getEffets();    // description des effets
}

// Implémente PotionDeBase : 50 PV, effet "Soin de base".           (1 pt)
```

```
// Implémente PotionDecorator (abstract) : délègue à la Potion wrappée. (1 pt)

// Implémente AvecAntidote extends PotionDecorator : (2 pts)
// +0 PV, ajoute l'effet "Antidote"

// Implémente AvecMana extends PotionDecorator : (2 pts)
// +30 PV, ajoute l'effet "Restauration de mana"
```

E.

Bridge — système de notification [6 pts]

Contexte. On veut envoyer des notifications (Urgente, Normale) via différents canaux (Email, SMS). Le Bridge découple le type du canal.

`src/main/java/partiel/Notification.java`

```
public interface CanalEnvoi { // (1 pt)
    void envoyer(String destinataire, String message);
}

// Implémente CanalEmail : affiche "[EMAIL] -> dest : message" (1 pt)
// Implémente CanalSMS : affiche "[SMS] -> dest : message" (1 pt)

public abstract class Notification { // (1 pt)
    protected CanalEnvoi canal;
    public Notification(CanalEnvoi canal) { this.canal = canal; }
    public abstract void notifier(String destinataire, String contenu);
}

// Implémente NotificationUrgente : (1 pt)
// préfixe le contenu avec "[URGENT] "
// Implémente NotificationNormale : (1 pt)
// envoie le contenu sans modification
```

F.

State — le personnage et ses états [7 pts]

Contexte. Un personnage peut être dans 3 états : Vivant, Empoisonné, Mort. Son comportement change selon l'état.

`src/main/java/partiel/EtatPersonnage.java`

```
public abstract class EtatPersonnage { // (1 pt)
    protected Personnage personnage;
    public abstract String attaquer(String cible);
    public abstract String recevoirPoison();
    public abstract String mourir();
}

// EtatVivant : (2 pts)
```

```
//  attaquer → "X attaque Y." | recevoirPoison → passe à Empoisonné
//  mourir   → passe à Mort

// EtatEmpoisonné : (2 pts)
//  attaquer → "X attaque Y. X perd 10 PV (poison)"
//  recevoirPoison → "Déjà empoisonné." | mourir → passe à Mort

// EtatMort : (2 pts)
//  toutes les actions → "Action impossible, X est mort."
```

REPONSES.MD §1.F

- 3.** Quelle est la différence entre le pattern State et un simple if/else sur un attribut enum ? Donne un avantage concret du State dans ce contexte.

PARTIE 02 Génériques, Lambdas & Streams*Types génériques · Interfaces fonctionnelles · Stream API · Optional* — **30 points****A.****Classe et méthodes génériques [8 pts]****1.** Implémente la classe `Paire<A,B>` qui stocke deux valeurs de types différents.`src/main/java/partie2/Paire.java`

```
public class Paire<A, B> {  
    // 1. Attributs + constructeur (1 pt)  
    // 2. getters getFirst(), getSecond() (1 pt)  
    // 3. swap() - retourne une Paire<B,A> avec les valeurs inversées (2 pts)  
    // 4. toString() (1 pt)  
}
```

2. Implémente les méthodes génériques suivantes dans `OutilsGeneriques` :`src/main/java/partie2/OutilsGeneriques.java`

```
public class OutilsGeneriques {  
  
    /** (2 pts) Retourne le max d'une List<T> où T est Comparable.  
     * Lève NoSuchElementException si la liste est vide. */  
    public static <T extends Comparable<T>> T max(List<T> liste) { }  
  
    /** (1 pt) Concatène les éléments d'une List<T> séparés par sep. */  
    public static <T> String concat(List<T> liste, String sep) { }  
}
```

B.**Interfaces fonctionnelles & lambdas [8 pts]****1.** Définis l'interface fonctionnelle `Transformation<T>` avec une méthode `appliquer(T valeur)` retournant `T`.`src/main/java/partie2/Transformation.java`

```
@FunctionalInterface  
public interface Transformation<T> {  
    T appliquer(T valeur); // (1 pt)  
}
```

2. Dans `LambdaFactory`, implémente les méthodes suivantes en utilisant des lambdas :

`src/main/java/partie2/LambdaFactory.java`

```
public class LambdaFactory {

    /** (2 pts) Retourne une Transformation<String> qui met en majuscules
     * et préfixe avec ">> " */
    public static Transformation<String> majusculeEtPrefixe() { }

    /** (2 pts) Retourne une Transformation<Integer> qui calcule n! (factorielle)
     */
    public static Transformation<Integer> factorielle() { }

    /** (3 pts) Retourne une Function<Integer,Integer> qui retourne
     * le n-ième terme de Fibonacci (F(0)=0, F(1)=1). */
    public static Function<Integer,Integer> fibonacci() { }

}
```

C.

Stream API [14 pts]

Contexte. On travaille sur une liste de produits d'une boutique en ligne :

```
public record Produit(String id, String nom, double prix, String categorie, int
stock) {}
```

Implémente toutes les méthodes ci-dessous en une chaîne Stream. Pas de boucle for.

`src/main/java/partie2/BoutiqueService.java`

```
public class BoutiqueService {
    private final List<Produit> produits;

    /** (2 pts) Produits en stock (stock > 0), triés par prix croissant. */
    public List<Produit> getProduitsDisponibles() { }

    /** (2 pts) Prix moyen des produits d'une catégorie donnée.
     * Retourne 0.0 si la catégorie est vide. */
    public double getPrixMoyen(String categorie) { }

    /** (3 pts) Map : catégorie → liste de noms de produits (triés alpha). */
    public Map<String, List<String>> getNomsParCategorie() { }

    /** (3 pts) Produits dont le stock est en dessous du seuil,
     * par catégorie : Map<categorie, nb de produits en rupture>. */
    public Map<String, Long> getAlertesRupture(int seuil) { }

    /** (2 pts) Optional<Produit> : le produit le plus cher de la boutique.
     * Utilise Stream + max(). */
    public Optional<Produit> getProduitLePlusCher() { }
```

```
/** (2 pts) Valeur totale du stock : somme de (prix * stock) pour chaque
produit. */
public double getValeurTotaleStock() { }
}
```

PARTIE 03 Réflexion & Annotations

Annotations custom · Introspection · Traitement par réflexion — 25 points

A.

Créer ses propres annotations [6 pts]

Définit les annotations suivantes qui serviront de base aux exercices B et C.

src/main/java/partie3/Entite.java

```
/** (2 pts) @Entite : applicable sur une classe, visible à l'exécution.
 * Attribut : String table() — nom de la table DB, défaut : "" */
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Entite { String table() default ""; }
```

src/main/java/partie3/Colonne.java

```
/** (2 pts) @Colonne : applicable sur un champ, visible à l'exécution.
 * Attributs : String nom() défaut "", boolean nullable() défaut true */
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Colonne { String nom() default ""; boolean nullable() default
true; }
```

src/main/java/partie3/Loggable.java

```
/** (2 pts) @Loggable : applicable sur une méthode, visible à l'exécution.
 * Attribut : String niveau() parmi "INFO", "WARN", "ERROR", défaut "INFO" */
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Loggable { String niveau() default "INFO"; }
```

B.

Introspection par réflexion [10 pts]

Contexte. La classe `Joueur` (fournie) utilise tes annotations :

```
@Entite(table = "joueurs")
public class Joueur {
    @Colonne(nom = "pseudo", nullable = false)
    private String pseudo;

    @Colonne(nom = "score")
    private int score;

    private String motDePasseHash;    // pas annoté – ne doit pas apparaître

    @Loggable(niveau = "WARN")
    public void resetScore() { this.score = 0; }

    public String getPseudo() { return pseudo; }
}
```

`src/main/java/partie3/Inspecteur.java`

```
public class Inspecteur {

    /** (2 pts) Retourne le nom de la table si @Entite est présent,
     *  sinon lève IllegalArgumentException. */
    public static String getNomTable(Class<?> clazz) { }

    /** (3 pts) Retourne les noms des champs annotés @Colonne.
     *  Si @Colonne.nom() est vide, utilise le nom du champ Java à la place. */
    public static List<String> getColonnes(Class<?> clazz) { }

    /** (2 pts) Retourne les noms des champs @Colonne où nullable = false. */
    public static List<String> getColonnesObligatoires(Class<?> clazz) { }

    /** (3 pts) Retourne les noms des méthodes annotées @Loggable
     *  dont le niveau correspond au paramètre donné (ex: "WARN"). */
    public static List<String> getMethodesLoggables(Class<?> clazz, String niveau)
    { }
}
```

C.

Générateur de requête SQL par réflexion [9 pts]

Contexte. En t'appuyant sur tes annotations, génère automatiquement des requêtes SQL à partir d'un objet Java.

`src/main/java/partie3/GenerateurSQL.java`

```
public class GenerateurSQL {

    /** (4 pts) Génère un SELECT : (4 pts)
     *  "SELECT pseudo, score FROM joueurs"
     *  Utilise @Entite.table() et les noms de @Colonne.
     *  Lève IllegalArgumentException si @Entite absent. */
}
```

```
public static String genererSelect(Class<?> clazz) { }

/** (5 pts) Génère un INSERT à partir d'une instance : (5 pts)
 * "INSERT INTO joueurs (pseudo, score) VALUES ('Alice', 1500)"
 * Lis les valeurs par réflexion (field.get(objet)).
 * Les champs non-nullables qui sont null lèvent IllegalStateException. */
public static String genererInsert(Object objet) throws Exception { }
}
```

REPONSES.MD §3.C

- 4.** Pourquoi faut-il appeler `field.setAccessible(true)` avant `field.get(objet)` pour les champs privés ?
- 5.** Cite un risque de sécurité lié à l'utilisation de `setAccessible()` en production.

PARTIE 04 I/O, Regex & REST*Fichiers · Expressions régulières · Mini-framework HTTP — 10 points***A.****Gestion de fichiers CSV [4 pts]**

Implémente un gestionnaire de sauvegarde de scores au format CSV (une ligne par joueur : pseudo,score).

src/main/java/partie4/ScoreManager.java

```
public class ScoreManager {

    /** (2 pts) Écrit la liste dans un fichier CSV. Utilise try-with-resources.
     *  Format : une ligne par entrée "pseudo,score\n" */
    public static void sauvegarder(String fichier, List<Paire<String,Integer>>
scores)
        throws IOException { }

    /** (2 pts) Lit le fichier et retourne la liste de paires.
     *  Ignore les lignes malformées (pas de crash). */
    public static List<Paire<String,Integer>> charger(String fichier)
        throws IOException { }

}
```

B.**Validation & extraction par regex [4 pts]****src/main/java/partie4/ValideurJeu.java**

```
public class ValideurJeu {

    /** (1 pt) Retourne true si le pseudo est valide :
     *  3 à 16 caractères, lettres, chiffres, tiret bas uniquement. */
    public static boolean pseudoValide(String pseudo) { }

    /** (2 pts) Extrait tous les scores d'un texte de la forme
     *  "Alice:1500 points, Bob:320 points"
     *  Retourne Map<String, Integer> : pseudo -> score. */
    public static Map<String, Integer> extraireScores(String texte) { }

    /** (1 pt) Remplace les suites de chiffres par "****" dans un texte.
     *  Ex: "Code 1234 et pin 5678" → "Code **** et pin ****" */
    public static String masquerNombres(String texte) { }

}
```

C.

Étendre le mini-framework REST [2 pts]

Contexte. Le projet contient un mini-framework REST maison basé sur les annotations `@Rest`, `@Get`, `@Post` et `@QueryParam`. Il démarre un serveur HTTP et route les requêtes par réflexion.

Ajoute un nouvel endpoint à ce framework :

`src/main/java/partie4/ScoreController.java`

```
@Rest(path = "/api/scores")
public class ScoreController {

    /** (1 pt) GET /api/scores/top?limit=5
     *  Retourne les limit meilleurs scores du ScoreManager sous forme de String.
     *  Format : "pseudo1:score1,pseudo2:score2,..." */
    @Get(path = "/top")
    public String getTop(@QueryParam(name = "limit") int limit) { }

    /** (1 pt) POST /api/scores/add
     *  Reçoit un ScoreDTO (pseudo + score), l'enregistre via ScoreManager.
     *  Retourne "OK" si succès. */
    @Post(path = "/add")
    public String add(ScoreDTO dto) { }
}
```

REPONSES.MD §4.C

- 6.** Le `RestEngine` lit les annotations `@Rest`, `@Get`, `@Post` via réflexion pour construire la table de routage. Explique en 3 lignes comment fonctionne ce mécanisme.

Bon courage !

CHECKLIST AVANT DE RENDRE

- `mvn test` passe sans erreur (ou tu expliques les échecs dans REPONSES.md).
 - `REPONSES.md` complet : toutes les sections renseignées.
 - ZIP nommé `NOM_Prenom_B2_Java.zip`.
 - Dépôt Git poussé + lien dans REPONSES.md.
 - Javadoc sur toutes les méthodes publiques implémentées.
-